Phase 1 - High-Performance Linear Algebra Kernels

4/5/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed

Details

High-Performance Linear Algebra Kernels

I would suggest to create a GitHub for your group and to provide the link.

Submission: Each team will submit a single repository containing:

* All source code files (.cpp and .h).

* A README.md file documenting the team members, build instructions, and answers to the discussion questions.

* A report (PDF format) detailing their findings and optimizations.

Team Size: 1-4 students per team.

Objective: To implement and optimize fundamental linear algebra operations (matrix-vector multiplication and matrix-matrix multiplication) in C++, focusing on performance considerations such as cache locality, memory alignment, and the impact of compiler optimizations like inlining. Teams will analyze the performance of their implementations using benchmarking and profiling tools.

Part 1: Baseline Implementations

Each team member will be responsible for implementing one of the following baseline functions using **raw pointers** for data storage and manipulation. Ensure your implementations are correct and handle dynamically allocated memory appropriately (remember to deallocate).

- **Team Member 1: Matrix-Vector Multiplication (Row-Major)**

  ```
  void multiply_mv_row_major(const double* matrix, int rows, int cols, c
  onst double* vector, double* result);
  ```

  Assume `matrix` is stored in row-major order.

- **Team Member 2: Matrix-Vector Multiplication (Column-Major)**

```
void multiply_mv_col_major(const double* matrix, int rows, int cols, const double* vector, double* result);
```

Assume `matrix` is stored in column-major order.

- **Team Member 3: Matrix-Matrix Multiplication (Naive)**

```
void multiply_mm_naive(const double* matrixA, int rowsA, int colsA, const double* matrixB, int rowsB, int colsB, double* result);
```

Assume `matrixA`, `matrixB`, and `result` are stored in row-major order.

- **Team Member 4: Matrix-Matrix Multiplication (Transposed B)**

```
void multiply_mm_transposed_b(const double* matrixA, int rowsA, int colsA, const double* matrixB_transposed, int rowsB, int colsB, double* result);
```

Assume `matrixA` and `result` are in row-major order, and `matrixB_transposed` is the transpose of matrix B (also in row-major order).

Requirements for Part 1:

- Each function should perform the correct mathematical operation.
- Allocate and deallocate memory for the input and output matrices/vectors within a test program.
- Include basic error handling (e.g., checking for null pointers, compatible dimensions).
- Write a simple test case for each function to verify its correctness.

Part 2: Performance Analysis and Optimization

Once the baseline implementations are complete, the team will work together to analyze and optimize the performance of these functions.

Tasks:

1. **Benchmarking:**
   - Develop a robust benchmarking framework using `std::chrono` to measure the execution time of each baseline function.
   - Test with various matrix and vector sizes (e.g., small, medium, large square matrices, and compatible vectors).
   - Perform multiple runs for each test case and calculate the average execution time and standard deviation.
   - Present your benchmarking results in a clear table or graph in your report.
2. **Cache Locality Analysis:**
   - For the matrix-vector multiplication implementations (row-major vs. column-major), analyze the cache access patterns. Explain which implementation is expected to perform better and

why, considering cache locality.
- For the matrix-matrix multiplication implementations (naive vs. transposed B), analyze how the memory access patterns differ and how the `transposed_b` approach might improve cache utilization.
- Design and run specific benchmark cases that highlight the impact of cache locality. For example, compare performance with different strides of data access.

3. **Memory Alignment:**
   - Investigate the impact of memory alignment on the performance of your matrix operations.
   - Modify your memory allocation to ensure that the matrices and vectors are aligned to a specific boundary (e.g., 64 bytes) using techniques like custom allocators or platform-specific alignment functions (you can also use an array).
   - Benchmark the aligned versions against the unaligned versions and report your findings. Did alignment provide a noticeable performance improvement? Under what conditions?
4. **Inlining:**
   - Experiment with the use of the `inline` keyword for small, frequently called helper functions within your matrix operations (if any).
   - Compile your code with and without aggressive compiler optimizations (e.g., `-O0` vs. `-O3` in GCC/Clang, `/Od` vs. `/O2` in MSVC).
   - Analyze how compiler optimizations and the `inline` keyword affect the performance. Discuss when inlining is likely to be beneficial and when it might not be (you can study the assembly code)
5. **Profiling:**
   - Use the appropriate profiling tools for your development environment:
     - **Linux:** `gprof` and `perf`.
       - **gprof:** Compile your code with the `-pg` flag. Run the executable. Analyze the generated `gmon.out` file using `gprof your_program gmon.out`. Understand the flat profile and call graph.
       - **perf:** Use commands like `perf stat ./your_program` to get summary statistics or `perf record -g ./your_program` followed by `perf report` for call graph analysis.
     - **macOS: Instruments** (part of Xcode).
       - Compile with debug symbols (`-g`). Open Instruments, choose the "Time Profiler" template, and select your executable as the target. Record the execution and analyze the "Call Tree" and "Top Functions" views. Alternatively, use the command-line tool `xcrun xctrace`.
     - **Windows: Visual Studio Performance Profiler**, **Windows Performance Toolkit (WPT)**, **or Cygwin with gprof**.

        - **Visual Studio:** Build in Release (with optional debug info). Use the Performance Profiler (Debug -> Performance Profiler or Alt+F2) and select the "CPU Usage" tool. Analyze the summary, functions, and call tree views.
        - **WPT:** Install the Windows ADK. Use Windows Performance Recorder (WPR) to collect a trace while running your application, and then analyze the trace using Windows Performance Analyzer (WPA), focusing on CPU usage graphs and call stacks.
        - **Cygwin (Optional):** If you have Cygwin installed (a Linux-like environment for Windows), you can compile your code using `g++ -pg your_program.cpp -o your_program` within the Cygwin terminal. Run the executable (`./your_program`). Then, use `gprof your_program gmon.out` to analyze the profiling data. Understand the flat profile and call graph as you would on Linux.
    - Profile the execution of your benchmarked code for at least one of the matrix multiplication implementations (both naive and transposed B).
    - Identify the parts of the code where the program spends the most time.
    - Analyze the profiler output (flat profile, call graph, or relevant views) and relate it to your understanding of the algorithms and their cache behavior. Include screenshots or relevant excerpts from the profiler output in your report.

6. **Optimization Strategies (Team Brainstorming and Implementation):**
    - Based on your analysis of cache locality, memory alignment, and profiling results, brainstorm and implement at least one significant optimization to one of your baseline matrix multiplication functions. This could involve:
        - **Loop reordering (for better cache locality).**
        - **Blocking/tiling (for improved cache reuse).**
        - **Other relevant optimization techniques discussed in class or found through research.**
    - Clearly document the optimization you implemented and the reasoning behind it.
    - Benchmark your optimized version against the baseline and report the performance improvement (if any).

Part 3: Discussion Questions (To be answered in the README.md)

1. Explain the key differences between pointers and references in C++. When would you choose to use a pointer over a reference, and vice versa, in the context of implementing numerical algorithms?
2. How does the row-major and column-major storage order of matrices affect memory access patterns and cache locality during matrix-vector and matrix-matrix multiplication? Provide specific examples from your implementations and benchmarking results.
3. Describe how CPU caches work (L1, L2, L3) and explain the concepts of temporal and spatial locality. How did you try to exploit these concepts in your optimizations?
4. What is memory alignment, and why is it important for performance? Did you observe a significant performance difference between aligned and unaligned memory in your experiments?

Explain your findings.

5. Discuss the role of compiler optimizations (like inlining) in achieving high performance. How did the optimization level affect the performance of your baseline and optimized implementations? What are the potential drawbacks of aggressive optimization?
6. Based on your profiling experience, what were the main performance bottlenecks in your initial implementations? How did your profiling results guide your optimization efforts?
7. Reflect on the teamwork aspect of this assignment. How did dividing the initial implementation tasks and then collaborating on analysis and optimization work? What were the challenges and benefits of this approach?

Keep in mind, this submission will count for everyone in your Q22026 Group group.

Choose a submission type

Text | Web URL | Upload | More

Submit Assignment